Introductory Programming and Problem-Solving Skills

Week - 6

Functions and Modules

Basanta Singh

Function

1.2 Introduction

Navigation

1. **Function**
   - **1.1 Motivation**
   - **1.2 Introduction**

- So far, we have limited ourselves to using only the most fundamental features of Python – variables, input/print and so on.
- In theory, these are the only instructions needed to write basic programs. From a ***practical* point-of-view**, however, these instructions alone are not enough.
- **Scenario** [**For Example**]: Facebook, contains over 10 million lines of code and it manage such **complexity, programs are divided into manageable pieces called program routines (or simply routines)**. Doing so, is a form of abstraction in which more general, less detailed view of a system can be achieved. In addition, program routines provides code reusability.

1.3 Program Routine In Python

Navigation

1. **Function**
   - **1.1 Motivation**
   - **1.2 Introduction**
   - **1.3 Function Routine**

- ***Program Routine*** in python can also be called as ***Function Routine***. Basically there are two types of function in python which are listed below:
  - **Standard Library Function:** These are built-in functions which we have using since from our first day at class. Some of the built in function that have been used are len, range, format, input, int and so on.
  - **User-defined Functions:** We can create our own functions based on our requirement.

1.3.1 What is Function Routine?

Navigation

1. **Function**
   - **1.1 Motivation**
   - **1.2 Introduction**
   - **1.3 Function Routine**
   - **1.3.1 What is Function Routine ?**

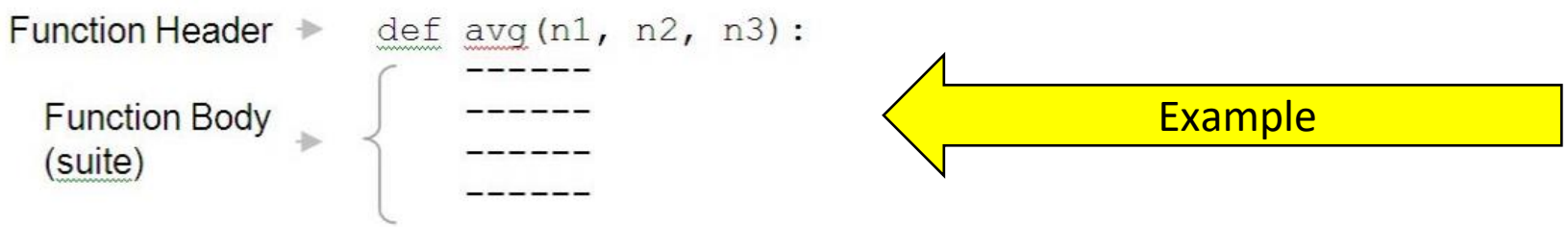
- A **routine** is a named group of instructions performing some task.
- A routine can be invoked (**called**) as many times as needed in a given program.
- When a routine terminates, execution automatically returns to the point from which it was called. Such routines may be predefined in the programming language, or designed and implemented by the programmer. A **function** is Python's version of a program routine.

1.3.2 How To Define a Function?

Navigation

1. **Function**
   - **1.1 Motivation**
   - **1.2 Introduction**
   - **1.3 Function Routine**
   - **1.3.1 What is Function Routine ?**
   - **1.3.2 How to Define a Function ?**

- A **function header** starts with the keyword **def**, followed by an identifier (e.g. avg), which is the function's name.
- The function name is followed by a comma-separated (possibly empty) list of identifiers (n1, n2, n3) called **parameters**. Following the *parameter list* is a colon (:)



- *Following the function header* is the **function body**, a suite (program block) containing the function's instructions. The statements within the suite must be **indented** at the same level, relative to the function header.

1.3.3 Let’s Understand Through Example

Navigation

1. Function

1.1 Motivation

1.2 Introduction

1.3 Function Routine

1.3.1 What is Function Routine ?

1.3.2 How to Define a Function ?

1.3.3 Example


```
def area_rectangle (length, breadth):
    areaOfRectangle = length * breadth
```

def: keyword used to declare a function

area_rectangle: name given for function

arguments: any value passed to a function

return (optional): returns value from a function

What is Indentation and why is it important?

1.3.3 Indentation in Python

Navigation

1. Function
   1.1 Motivation
   1.2 Introduction
   1.3 Function Routine
   1.3.1 What is Function Routine ?
   1.3.2 How to Define a Function ?
   1.3.3 Indentation in Python

- One fairly unique aspect of Python is that the amount of indentation of each line is significant.
- In most programming languages, indentation has no effect on program logic, it is simply used to align program lines to aid readability.
- In Python, however, indentation is used to associate and group statements.

```
def avg (n1, n2, n3):
        <indented code block>
<not-indented statement>
```

1.3.3 Let's Understand Through Example

Navigation

1. Function

1.1 Motivation

1.2 Introduction

1.3 Function Routine

1.3.1 What is Function Routine ?

1.3.2 How to Define a Function ?

1.3.3 Example

Indentation

```
def area_rectangle (length, breadth):
    areaOfRectangle = length * breadth
```

def: keyword used to declare a function

area_rectangle: name given for function

arguments: any value passed to a function

return (optional): returns value from a function

1.3.3 Actual Arguments

Navigation

1. Function
   1.1 Motivation
   1.2 Introduction
   1.3 Function Routine
   1.3.1 What is Function Routine ?
   1.3.2 How to Define a Function ?
   1.3.3 Actual Arguments

- **Definition** [**Arguments**]: The number of items in a parameter list indicates the number of values that must be passed to the function, called actual arguments (or simply "arguments"), such as the variables below:

```
>>> num_1 = 10
>>> num_2 = 25
>>> num_3 = 16
>>> avg (num_1, num_2, num_3)
```

Remember : Functions are generally defined at the top of a program. However, every function must be defined before it is called, otherwise the interpreter will not recognize it.

1.3.3 Let's Understand Through Example

Navigation

1. Function

1.1 Motivation

1.2 Introduction

1.3 Function Routine

1.3.1 What is Function Routine ?

1.3.2 How to Define a Function ?

1.3.3 Example


```
def area_rectangle (length, breadth):
    areaOfRectangle = length * breadth
```

Arguments

def: keyword used to declare a function

area_rectangle: name given for function

arguments: any value passed to a function

return (optional): returns value from a function

Two Types of Function!!

School Days Function!!

Navigation

1. Function
   1.1 Motivation
   1.2 Introduction
   1.3 Function Routine
   1.4 Function Types
   1.4.1 Value Returning Function
   School Days Function

Value Returning Function: A value-returning function is a program routine (function) called for its return value, and is therefore similar to a mathematical function, e.g.,

f(x) = 2x

In this notation, “x” stands for any numeric value that function f may be applied to,

f(2) = 2 x 2 = 4

Program functions are similarly used.

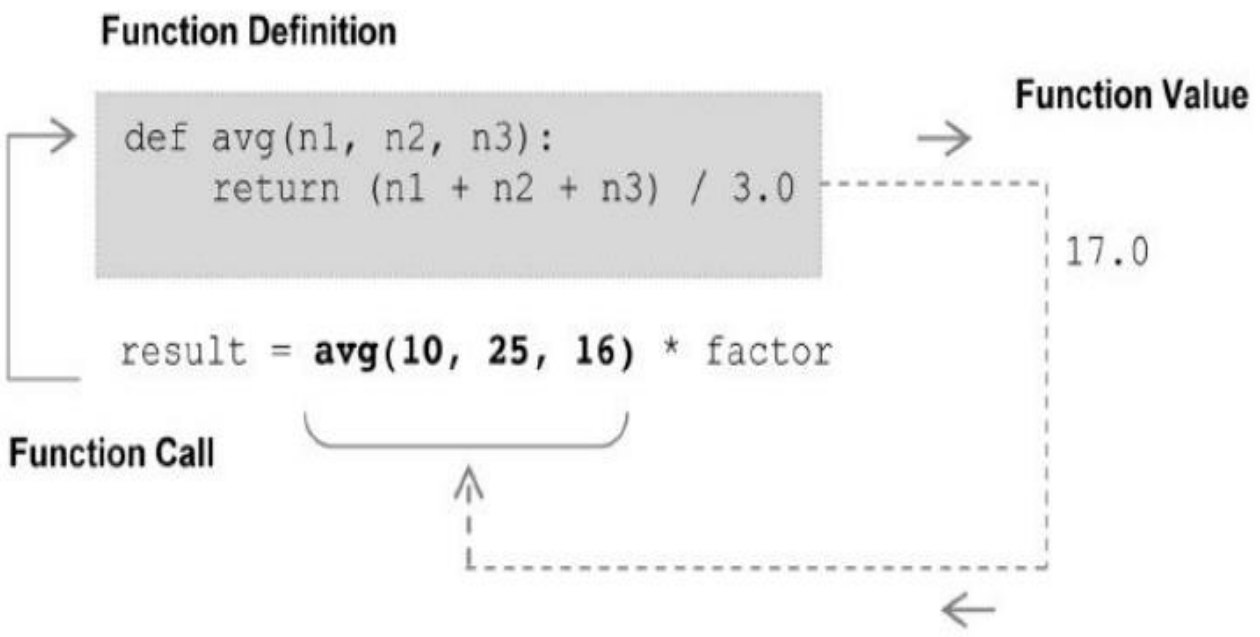
Let’s Try:

Let’s say there is a function named f(x), and the value you want to pass is 45. What will be the value of f(x) = 2x + 5

1.4.1 Value Returning Functions

Navigation

1. Function
   - 1.1 Motivation
   - 1.2 Introduction
   - 1.3 Function Routine
   - 1.4 Function Types
   - 1.4.1 Value Returning Function



Function avg takes three arguments (n1, n2, and n3) and returns the average of the three.

Explanation [FYI]: The function call **avg(10, 25, 16)**, therefore, is an expression that evaluates to the returned function value. This is indicated in the function's return statement of the form **return expr**, where **expr** may be any expression.

1.4.1.1 Let's Understand Through Example

Navigation

1. Function
   1.1 Motivation
   1.2 Introduction
   1.3 Function Routine
   1.4 Function Types
   1.4.1 Value Returning Function
   1.4.1.1 Example


```
def area_rectangle(length, breadth):
    areaOfRectangle = length * breadth
    return areaOfRectangle


result = area_rectangle(10, 20)
print(result)
```

Returns value to **result**

def: keyword used to declare a function

area_rectangle: name given for function

arguments: any value passed to a function

return (optional): returns value from a function

What If We Don't Return?

Navigation

1. Function

1.1 Motivation

1.2 Introduction

1.3 Function Routine

1.4 Function Types

1.4.1 Value Returning Function

1.4.1.1 Example

- **Try without returning value.**

```
def area_rectangle(length, breadth):
    areaOfRectangle = length * breadth


result = area_rectangle(10, 20)
print(result)
```

Without return type

1.4.1.2 Let's Try!!

Navigation

1. Function
1.1 Motivation
1.2 Introduction
1.3 Function Routine
1.4 Function Types
1.4.1 Value Returning Function
1.4.1.1 Example
1.4.1.2 Try out

- **From the Python Shell,** let's enter the following **function,** making sure to indent the code as given. We Hit return twice after the last line of the function is entered.

```
>>> def avg(n1, n2, n3):
        return (n1 + n2 + n3) / 3.0
```

Next, let's enter the following function calls and observe the results.

```
>>> avg(10, 25, 40)
>>> avg(40, 10, 25)
>>> avg(40, 25, 10)
```

1.4.2 Non-Value Returning Functions

Navigation

1. Function
   1.1 Motivation
   1.2 Introduction
   1.3 Function Routine
   1.4 Function Types
   1.4.1 Value Returning Function
   1.4.2 Non-value Returning Functions

- A **non-value-returning function** is called not for a returned value, but for its *side effects*.
- **Side Effect** [**Definition**]: A side effect is an action other than returning a function value, such as displaying output.

Function Definition

```
def displayWelcome():
    print('This program will convert between Fahrenheit and Celsius')
    print('Enter (F) to convert Fahrenheit to Celsius')
    print('Enter (C) to convert Celsius to Fahrenheit')

# main
.
displayWelcome()
```

1.4.2.1 Let's Understand Through Example

Navigation

1. Function
   - 1.1 Motivation
   - 1.2 Introduction
   - 1.3 Function Routine
   - 1.4 Function Types
   - 1.4.1 Value Returning Function
   - 1.4.2 Non-value Returning Functions
   - 1.4.2.1 Example


```python
def welcome():
    print("\nWelcome to a system where area of rectangle is calculated \nOther calculation Coming Soon..\n")


def area_rectangle(length, breadth):
    areaOfRectangle = length * breadth
    return areaOfRectangle


welcome()
result = area_rectangle(10, 20)
print("The area of rectangle is: ", result)
```

Defining non-value returning function

```
Welcome to a system where area of rectangle is calculated
Other calculation Coming Soon..

The area of rectangle is:  200
```

1.4.2.2 Let's Try!!

Navigation

1. Function
1.1 Motivation
1.2 Introduction
1.3 Function Routine
1.4 Function Types
1.4.1 Value Returning Function
1.4.2 Non-value Returning Functions
1.4.2.1 Example
1.4.2.2 Try out

From the Python Shell, let's enter the following function, making sure to indent the code as given. Then we'll make the following function calls and observe the results.

```
>>> def hello(name):
        print('Hello', name + '!')

>>> name = 'John'
>>> hello(name)
```

1.4.4 Guess The Topic

Navigation

1. Function

1.1 Motivation

1.2 Introduction

1.3 Function Routine

1.4 Function Types

1.4.1 Value Returning Function

1.4.2 Non-value Returning Functions

1.4.3 Function Calls Overview

1.4.4 Guess The Topic

+

How and Why To Call Function?

1.4.1.3 Calling Value Returning Functions

Navigation

1. Function

1.1 Motivation

1.2 Introduction

1.3 Function Routine

1.4 Function Types

1.4.1 Value Returning Function

1.4.1.1 Example

1.4.1.2 Try out

1.4.1.3 Calling Value Returning Functions

- Calls to value-returning functions can be used anywhere that a function's return value is appropriate, **for example:**

```
result = max(num_list) * 100
```

Note: Function is called by function_name and parenthesis

Match the Following!!

Navigation

1. Function
1.1 Motivation
1.2 Introduction
1.3 Function Routine
1.4 Function Types
1.4.1 Value Returning Function
1.4.1.1 Example
1.4.1.2 Try out
1.4.1.3 Calling Value Returning Functions
(Match the Following)

Some of the scenario where function could be called at:

- result = max(num_list1) * max(num_list2)
- result = abs(max(num_list))
- if max(num_list) < 10: …
- print('Largest value in num_list is ', max(num_list))

- Function call as conditional expression
- Expressions containing multiple function calls
- Function call as part of calls to built-in print function
- Function call as an argument to another function call

1.4.1.3 Calling Value Returning Functions

Navigation

1. Function
1.1 Motivation
1.2 Introduction
1.3 Function Routine
1.4 Function Types
1.4.1 Value Returning Function
1.4.1.1 Example
1.4.1.2 Try out
1.4.1.3 Calling Value Returning Functions

- A value-returning function may return more than one value by returning the values as a tuple.

Function definition:

```
def maxmin(num_list):
    return (max(num_list), min(num_list))
```

Function use:

```
weekly_temps = [45, 30, 52, 58, 62, 48, 49]
```

(a) highlow_temps = maxmin(weekly_temps)

(b) high, low = maxmin(weekly_temps)

1.4.2.3 Calling Non- Value Returning Functions

Navigation

1. **Function**
   - **1.1 Motivation**
   - **1.2 Introduction**
   - **1.3 Function Routine**
   - **1.4 Function Types**
     - **1.4.1 Value Returning Function**
     - **1.4.2 Non-value Returning Functions**
       - **1.4.2.1 Example**
       - **1.4.2.2 Try out**
       - **1.4.2.3 Calling Non-value Returning Functions**

- Calls to non-value-returning functions are for the ***side-effects*** produced, and not for a returned function value, such as displaying output on the screen:

```
display_welcome()
```

- It does not make any sense to treat this function call as an expression,

```
welcome_displayed = display_welcome()
```

As shown in the examples above, both value-returning and non-value-returning functions can be designed to take zero or more arguments.

Let's Understand Through Example

Navigation

1. Function
   1.1 Motivation
   1.2 Introduction
   1.3 Function Routine
   1.4 Function Types
   1.4.1 Value Returning Function
   1.4.2 Non-value Returning Functions
   1.4.2.1 Example
   1.4.2.2 Try out
   1.4.2.3 Calling Non-value Returning Functions (Example)

```python
def welcome():
    print("\nWelcome to a system where area of rectangle is calculated \nOther calculation Coming Soon..\n")


def area_rectangle(length, breadth):
    areaOfRectangle = length * breadth
    return areaOfRectangle


welcome()
result = area_rectangle(10, 20)
print("The area of rectangle is: ", result)
```

Calling function

```
Welcome to a system where area of rectangle is calculated
Other calculation Coming Soon..

The area of rectangle is:  200
```

1.4.3 Function Calls Overview

Navigation

1. Function
1.1 Motivation
1.2 Introduction
1.3 Function Routine
1.4 Function Types
1.4.1 Value Returning Function
1.4.2 Non-value Returning Functions
1.4.3 Function Calls Overview

A call to a value-returning function is an expression. It evaluates to the value returned by the function call. Thus, calls to value-returning functions are made part of a larger expression:

```
result = avg(10, 25, 16) * factor
```

A call to a non-value-returning function is a statement. Thus, calls to non-value-returning functions are written as a statement (instruction) on its own:

```
displayWelcome()
```

Note: Technically, all functions in Python are value-returning, since functions that do not return a value return special value None

Let's Try!!

Navigation

1. Function
   1.1 Motivation
   1.2 Introduction
   1.3 Function Routine
   1.4 Function Types
1.4.2 Non-value Returning Functions
   1.4.2.1 Example
   1.4.2.2 Try out
   1.4.2.3 Calling Non-value Returning Functions
(Try Out)

Let's enter the definition of the functions given below, then enter the following function calls and observe the results.

```
>>> def say_hello():
        print('Hello!')

>>> say_hello()
???
>>> t = say_hello()
???
>>> t
???
>>> t == None
???
```

```
>>> def build_hello(name):
        return 'Hello' + name + '!'

>>> greeting = build_hello('Charles')
>>> print(greeting)
???

>>> build_hello('Charles')
???

>>> build_hello()
???
```

1.5 Parameter Passing

Navigation

1. **Function**
   - **1.1 Motivation**
   - **1.2 Introduction**
   - **1.3 Function Routine**
   - **1.4 Function Types**
   - **1.5 Parameter Passing**

Definition [**Parameter Passing**]: Parameter passing is the process of passing arguments to a function. Recall that *actual arguments* are the values passed to a function's *formal parameters*.

- **Formal parameters:** These are the parameters that are listed in the definition of a function or procedure. They act as placeholders for the actual values that will be passed to the function or procedure when it is called.

- **Actual parameters:** These are the values that are passed to a function or procedure when it is called. They are also known as **arguments**.

Note: The correspondence of actual arguments and formal parameters is ***determined by the order of the arguments passed***, and not their names.

Let's visualize

Navigation

1. **Function**
   - **1.1 Motivation**
   - **1.2 Introduction**
   - **1.3 Function Routine**
   - **1.4 Function Types**
   - **1.5 Parameter Passing (Visualization)**

The following parameter passing is perfectly valid.

It is fine to **pass** *actual arguments* num1 and num2 to function ordered as shown (either num1 as the first argument, or num2 as the first)

```
def ordered(n1, n2):  → 1, 2 (Approved)
        return n1 > n2

num1 = int(input('Enter a number: '))
num2 = int(input('Enter another number: '))
ordered(num1, num2) → 1, 2 (Good To Go!!)
ordered(num2, num1) → 1, 2 (Good To Go!!)
```

1.5.1 Positional Argument

Navigation

1. **Function**
   - **1.1 Motivation**
   - **1.2 Introduction**
   - **1.3 Function Routine**
   - **1.4 Function Types**
   - **1.5 Parameter Passing**
   - **1.5.1 Positional Argument**

- The functions we have looked at so far were called with a fixed number of *positional arguments*.
- **Definition [Positional Argument]:** A positional argument is an argument that is assigned to a particular parameter based on its position in the argument list,

```
def mortgage_rate(amount, rate, term)

monthly_payment = mortgage_rate(350000, 0.06,  20)
```

1.5.1.1 Let’s Understand Through Example

Navigation

1. Function
   1.1 Motivation
   1.2 Introduction
   1.3 Function Routine
   1.4 Function Types
   1.5 Parameter Passing
   1.5.1 Positional Argument
   1.5.1.1 Example


```python
def welcome():
    print("\nWelcome to a system where area of rectangle is calculated \nOther calculation Coming Soon..\n")


def area_rectangle(length, breadth):
    areaOfRectangle = length * breadth
    return areaOfRectangle


welcome()
result = area_rectangle(10, 20)
print("The area of rectangle is: ", result)
```

```
Welcome to a system where area of rectangle is calculated
Other calculation Coming Soon..

The area of rectangle is:  200
```

1.5.2 Keyword Argument

Navigation

1. Function
   1.1 Motivation
   1.2 Introduction
   1.3 Function Routine
   1.4 Function Types
   1.5 Parameter Passing
   1.5.1 Positional Argument
   1.5.2 Keyword Argument

- However, python provides the option of calling any function by the use of ***keyword arguments***.
- **Definition [Keyword Argument]:** A keyword argument is an argument that is specified by name, rather than as a positional argument.

```
def mortgage_rate(amount, rate, term)

monthly_payment = mortgage_rate(rate=0.06, term=20, amount=350000)
```

Remember: This can be a useful way of calling a function if it is easier to remember the parameter names than it is to remember their order.

1.5.2.1 Let’s Understand Through Example

Navigation

1. Function
   1.1 Motivation
   1.2 Introduction
   1.3 Function Routine
   1.4 Function Types
   1.5 Parameter Passing
   1.5.1 Positional Argument
   1.5.2 Keyword Argument
   1.5.2.1 Example


```python
def welcome():
    print("\nWelcome to a system where area of square is calculated \nOther calculation Coming Soon..\n")

def area_square(length, breadth):
    area_of_square = length * length
    print("The value of breadth", breadth)
    return area_of_square


welcome()
print("The are of square is: ", area_square(breadth=50, length=10))
```

```
Welcome to a system where area of square is calculated
Other calculation Coming Soon..

The value of breadth 50
The are of square is:  100
```

1.5.3 Using Both Argument At Same Time?

Navigation

1. Function
   1.1 Motivation
   1.2 Introduction
   1.3 Function Routine
   1.4 Function Types
   1.5 Parameter Passing
   1.5.1 Positional Argument
   1.5.2 Keyword Argument
   1.5.3 Using Both Argument at Same Time ?

- It is possible to call a function using both positional and keyword arguments. However, all positional arguments must come before all keyword arguments in the function call

```
def mortgage_rate(amount, rate, term)

monthly_payment = mortgage_rate(35000, term=20, rate=0.06)
```

Remember: This form of function call might be useful, for example, if you remember that the first argument is the loan amount, but you are not sure of the order of the last two arguments rate and term.

1.5.3.1 Let's Try!!

Navigation

1. **Function**

1.5 Parameter Passing

1.5.1 Positional Argument

1.5.2 Keyword Argument

1.5.3 Using Both Argument at Same Time ?

1.5.3.1 Try Out

Let's enter the following function definition in the Python Shell.

Execute the statements below and observe the results.

```
def mortgage_rate(amount, rate, term):
    total  = amount * (1+ (rate/100) * term)
    print(('${0:.2f} at {1:.2f}% over {2:} years is ${3:.2f}'.format(amount
```

```
>>> mortgage_rate(50000, 3.5, 20)
???
>>> mortgage_rate(amount=50000, rate=3.5, term=20)
???
>>> mortgage_rate(rate=3.5, term=20, amount=50000)
???
```

1.5.4 Default Arguments

Navigation

1. Function
1.5 Parameter Passing
1.5.1 Positional Argument
1.5.2 Keyword Argument
1.5.3 Using Both Argument at Same Time ?
1.5.4 Default Arguments

- Python provides for ***default arguments***.
- **Definition [Default Argument]:** A *default argument* is an argument that can be optionally provided.

```
def mortgage_rate(amount, rate, term=20)

monthly_payment = mortgage_rate(35000, 0.62)
```

Parameter ***term*** is assigned a default value, 20, and therefore is optionally provided when calling function ***mortgage_rate*.**

Remember: All positional arguments must come before any default arguments in a function definition.

1.5.4.1 Let's Try!!

Navigation

1. **Function**

1.5 Parameter Passing

1.5.1 Positional Argument

1.5.2 Keyword Argument

1.5.3 Using Both Argument at Same Time ?

1.5.4 Default Arguments

1.5.4.1 Try Out

Let's enter the following function definition in the Python Shell. Execute the statements below and observe the results.

```
def mortgage_rate(amount, rate, term = 20):
    total  = amount * (1+ (rate/100) * term)
    print(('${0:.2f} at {1:.2f}% over {2:} years is ${3:.2f}'.format(amount
```

```
>>> mortgage_rate(50000, 3.5)
???
>>> mortgage_rate(amount=50000, rate=3.5, 10)
???
>>> mortgage_rate(amount = 50000, rate = 3.5)
???
```

Variable Scope

Local Variables

Navigation

1. Function
1.5 Parameter Passing
1.5.1 Positional Argument
1.5.2 Keyword Argument
1.5.3 Using Both Argument at Same Time ?
1.5.4 Default Arguments
1.5.4.1 Try Out

- Variable Scope has to do with the parts a program that a given variable is accessible.
- **Definition** [**Local Variable**]: A local variable is a variable that is only accessible from within a given function. Such variables are said to have local scope

What does this mean then?

- Local variables are only accessible within the inside function they are defined within.

1.5.4.2 Let's Understand Through Example

Navigation

1. Function

1.5 Parameter Passing

1.5.1 Positional Argument

1.5.2 Keyword Argument

1.5.3 Using Both Argument at Same Time ?

1.5.4 Default Arguments

1.5.4.1 Try Out

1.5.4.2 Example


```
def func1():
    n = 10
    print('n in func1 = ', n)
```

```
def func2():
    n = 20
    print('n in func2 before call to func1 = ', n)
    func1()
    print('n in func2 after call to func1 = ', n)
```

```
>>> func2()
n in func2 before call to func1 =  20
n in func1 =  10
n in func2 after call to func1 =  20
```

1.5.5 Explanation – Example Local Variable

Navigation

1. Function

1.5 Parameter Passing

1.5.5 Local Variable

Explanation: Both func1 and func2 contain identifier n.

Function func1 assigns n to 10, while function func2 assigns n to 20.

Both functions display the value of n **when called—func2** displays the value of n both before and after its call to func1.

Question: If identifier n represents the same variable, then shouldn't its value change to 10 after the call to func1?

The execution of func2, however, shows that the value of n remains unchanged.

1.5.5.1 Can I Reuse My Local Variable?

Navigation

1. Function

1.5 Parameter Passing

1.5.5 Local Variable

1.5.5.1 Can I Reuse My Local Variable ?

Now consider the following:

```
def func1():
    # n = 10
    print('n in func1 = ', n)
```

```
def func2():
    n = 20
    print('n in func2 before call to func1 = ', n)
    func1()
    print('n in func2 after call to func1 = ', n)
```

```
>>> func2()
n in func2 before call to func1 =  20
Traceback (most recent call last):
  .
  .
    print('n in func1 = ', n)
NameError: global name 'n' is not defined
```

In this case, when func2 is called, we get an error that variable n is not defined within func1. **This is because variable n defined in func2 is inaccessible from func1.**

1.5.5.2 How Long Variable Live?

Navigation

1. Function

1.5 Parameter Passing

1.5.5 Local Variable

1.5.5.1 Can I Reuse My Local Variable ?

1.5.5.2 How Long Variable Live ?

- **Concept [Variable Lifetime]:** The period of time that a variable exists is called its lifetime.
- **Local variables are automatically created (allocated memory) when a function is called and destroyed (deallocated) when the function terminates.**
- Thus, the lifetime of a local variable is equal to the duration of its function's execution. Consequently, the values of local variables are not retained from one function call to the next.

1.5.5.3 Let's Try!!

Navigation

1. Function

1.5 Parameter Passing

1.5.5 Local Variable

1.5.5.1 Can I Reuse My Local Variable ?

1.5.5.2 How Long Variable Live ?

1.5.5.3 Try Out

Let's enter the following function definition in the Python shell. Execute the statements below and observe the results.

```
>>> def func1():
        some_var = 10
```

```
>>> func1()
>>> some_var
???
```

1.5.6 Global Variable and Global Scope

Navigation

1. **Function**

1.5 Parameter Passing

1.5.5 Local Variable

1.5.6 Global Variable and Global Scope

Definition [Global Variable]: A global variable is a variable that is defined outside of any function definition. Such variables are said to have global scope.

```
maxm = 100  # Global Variable


def func1(count):
    count = count + maxm
    print("The value of count at function 1: ", count)


def func2(count):
    count = count + maxm
    print("The value of count at function 2: ", count)


func1(100)
func2(200)
```

Accessing Global Variable

1.5.6.1 Explanation – Example Global Variable

Navigation

1. Function

1.5 Parameter Passing

1.5.5 Local Variable

1.5.6 Global Variable and Global Scope

1.5.6.1 Explanation - Example

- **Variable max is defined outside func1 and func2, and therefore "global" to each**. Thus, it is referred to as a **global variable.** As a result, it is directly accessible by both functions.

➢ **The use of global variables is generally considered to be bad programming style**. Although it provides a convenient way to share values among functions, all functions within the scope of a global variable can access and alter it. This may include functions that have no need to access the variable, but none-the-less may unintentionally alter it.

➢ **Another reason that the use of global variables is bad practice is related to code reuse**. If a function is to be reused in another program, the function will not work properly if it is reliant on the existence of global variables that are non existent in the new program

UNIVERSITY OF WOLVERHAMPTON

BIRATNAGAR INTERNATIONAL COLLEGE

ing

BONUS topic

2. Docstrings

Navigation

1. **Function**
2. **Docstrings**

Python docstrings are string literals that appear right after the definition of a function, method, class, or module. Let's look at an example:

```
def square(n):
    '''Takes in a number n, returns the square of n'''
    return n**2
```

Inside the triple quotation marks is the docstring of the function square() as it appears right after its definition. We can see that It is used to document the code.

Remember: Triple quotation marks can also be used to create multi-line strings.

2.1 Single Line Docstrings

Navigation

1. Function
2. Docstrings
2.1 Single Line Docstrings

Single line docstrings fit on one line

```
def square(n):
    '''Takes in a number n, returns the square of n'''
    return n**2
```

Standard conventions to write single-line docstrings:

- Even though they are single-lined, we still use the triple quotes around these docstrings
- The closing quotes are on the same line as the opening quotes.
- There's no blank line either before or after the docstring.
- They should not be descriptive, rather they must follow "Do this, return that" structure

2.2 Multi-Line Docstrings

Navigation

1. Function
2. Docstrings
2.1 Single Line Docstrings
2.2 Multi-Line Docstrings

- Multi-line docstrings consist of a summary line just like a one-line docstring, followed by a blank line, followed by a more elaborate description.
- The PEP 257 document provides the standard conventions to write multi-line docstrings. https://www.python.org/dev/peps/pep-0257/
- The docstring for a function or method should summarize its behaviour and document its arguments and return values. It should also list all the exceptions (more on this at a later date) that can be raised and other optional arguments.

2.2.1 Multi-Line Example

Navigation

1. Function
2. Docstrings
2.1 Single Line Docstrings
2.2 Multiline Docstrings
2.2.1 Multi-Line Example


```python
def add_decimals(a, b):
    '''
    Returns the sum of two decimal numbers.

        Parameters:
            a (int): A decimal integer
            b (int): Another decimal integer

        Returns:
            sum (str): Decimal string of the sum of a and b
    '''
    return str(a + b)

print(add_decimals.__doc__)
```

Remember: Docstrings can be accessed programmatically using the __doc__ attribute.

Important: You will be expected to document your functions when submitting your assessments!

UNIVERSITY OF WOLVERHAMPTON

BIRATNAGAR INTERNATIONAL COLLEGE

ing